

Università degli Studi di Salerno
Corso di Ingegneria del Software

FakeNews Checker
Test Plan
Versione 1.0



FakeNews
Checker

Data: 20/12/2025

Progetto: FakeNews Checker	Versione: 1.0
Documento: Test Plan	Data: 20/12/2025

Coordinatore del progetto:

Nome	Matricola

Partecipanti:

Nome	Matricola
Swami Falasca	0512119323
Federica De Simone	0512120472
Giuseppe Tirelli	0512120367

Scritto da:	Swami Falasca, Federica De Simone, Giuseppe Tirelli
--------------------	---

Revision History

Data	Versione	Descrizione	Autore
20/12/2025	1.0	Prima versione del TP	Swami Falasca, Federica De Simone, Giuseppe Tirelli

Indice

1.INTRODUZIONE	4
1.1. References	4
2. SYSTEM OVERVIEW	4
3. FEATURES TO BE TESTED/NOT TO BE TESTED	6
4. PASS/FAIL CRITERIA	7
5. APPROACH	7
6. SUSPENSION AND RESUMPTION	8
7. TESTING MATERIALS	9
8. TEST CASES	9
9. TESTING SCHEDULE	10
10. DEFINITIONS, ACRONYMS AND ABBREVIATIONS	10

1.INTRODUZIONE

Il presente documento di **Test Plan** definisce le strategie, i metodi e le risorse necessarie per eseguire le attività di testing del sistema **FakeNews Checker**. L'obiettivo principale del piano di test è garantire che il software soddisfi pienamente i requisiti funzionali e non funzionali definiti nel documento **RAD_FakeNewsChecker**. Il piano di test mira a individuare eventuali errori, malfunzionamenti o bug attraverso test di unità, di integrazione e di sistema, al fine di fornire un prodotto stabile, affidabile e conforme alle aspettative del cliente.

FakeNews Checker è una piattaforma web progettata per contrastare la disinformazione online, consentendo la consultazione di notizie, la segnalazione di articoli e la gestione delle verifiche da parte di gestori. Il sistema è suddiviso in più sottosistemi (gestione utenti, notizie, segnalazioni, verifiche e infrastruttura tecnica) che richiedono attività di testing specifiche per garantire la correttezza delle funzionalità e delle interazioni tra i componenti. Le attività di testing verranno svolte adottando approcci **black-box** e **bottom-up**, verificando sia i singoli componenti sia il corretto funzionamento complessivo del sistema.

1.1. References

Per la stesura del presente documento è stato preso come riferimento il libro di testo: “Object-Oriented Software Engineering: Using UML, Patterns, and Java” – Third Edition, di Bernd Bruegge e Allen H. Dutoit.

Documenti di progetto:

- **RAD_FakeNewsChecker** (Requirements Analysis Document)
Documento contenente i requisiti funzionali e non funzionali del sistema.
- **SDD_FakeNewsChecker** (System Design Document)
Documento che descrive l'architettura del sistema e i sottosistemi individuati.

2. SYSTEM OVERVIEW

FakeNews Checker è una piattaforma web centralizzata per il contrasto alla disinformazione. La sua architettura, come delineata nel System Design

Document (SDD), si basa su un **modello three-tier** che separa nettamente i livelli di presentazione, logica di business e persistenza dei dati. Questa suddivisione consente di migliorare la manutenibilità e la modularità del sistema, rendendolo più facile da testare e aggiornare.

1. **Presentation Layer:** dedicato all'interazione con l'utente attraverso un'interfaccia web responsive realizzata con tecnologie come JSP, HTML5, CSS e JavaScript, per la consultazione delle notizie, segnalazioni, gestione profilo e pannelli gestionali.
2. **Application Layer:** rappresenta la logica di business. Essa serve da intermediario tra il Presentation Layer e il Data Layer che gestisce autenticazione, segnalazioni, verifiche, ricerche e gestione contenuti.
3. **Data Layer:** si occupa della persistenza dati tramite database relazionale MySQL per archiviare le informazioni relative alle entità presenti nel database gestito tramite classi DAO (es. NotiziaDAO, SegnalazioneDAO, UtenteDAO, etc.).

Il Presentation Layer invia richieste al Application Layer, che le elabora e interagisce con il Data Layer.

3. FEATURES TO BE TESTED/NOT TO BE TESTED

➤ Utente non registrato

- Consultazione delle notizie
- Visualizzazione dello stato di verifica
- Ricerca per titolo
- Accesso alla pagina di registrazione/login

➤ Utente registrato

- Login e Logout
- Segnalazione di notizie sospette
- Invio articoli per verifica
- Visualizzazione storico segnalazioni
- Modifica dati personali

➤ Gestore delle Verifiche

- Visualizzazione segnalazioni
- Verifica delle notizie
- Aggiornamento stato articoli
- Rimozione di notizie non attendibili

➤ Gestore Tecnico

- Monitoraggio database
- Gestione errori

- Aggiornamenti del sistema

4. PASS/FAIL CRITERIA

L'attività di testing ha lo scopo di accertare che il sistema FakeNews Checker rispetti in modo completo sia i requisiti funzionali sia quelli non funzionali. Ogni componente del sistema viene messa alla prova per verificare che il suo comportamento sia corretto e che gli output generati risultino coerenti con gli input ricevuti. L'esecuzione dei test prevede il confronto tra ciò che il sistema dovrebbe produrre e ciò che effettivamente produce: quando i due valori coincidono, il test è considerato superato; se emergono differenze, il test viene invece segnato come fallito.

L'analisi degli errori si basa sull'utilizzo di un oracolo, ossia un riferimento che descrive il comportamento atteso del sistema. Qualora una funzionalità non rispetti tali aspettative, sarà necessario intervenire sul codice, correggere le anomalie rilevate, rieseguire i test e assicurarsi che tutte le specifiche richieste risultino finalmente soddisfatte.

5. APPROACH

L'attività di testing prevista per FakeNews Checker è articolata in tre momenti fondamentali.

1. **Test unitario**, dedicato alla verifica delle singole classi e dei metodi che compongono il sistema. In questo contesto viene adottato un approccio di tipo black-box, concentrato sugli input forniti e sugli output prodotti, senza analizzare la logica interna delle componenti. Per gestire in modo efficace la varietà dei possibili dati in ingresso, viene applicata la tecnica della *category partition*, che suddivide gli input in insiemi organizzati, ciascuno rappresentativo di una specifica classe di equivalenza. Questo metodo consente di individuare rapidamente eventuali malfunzionamenti nelle unità testate, che vengono poi corrette e sottoposte nuovamente a verifica per assicurarne la piena aderenza ai requisiti.
2. **Test di integrazione**, finalizzato a controllare che le diverse parti del sistema collaborino correttamente tra loro. L'obiettivo è accertare che i moduli, una volta combinati, interagiscano senza generare errori. In

questa fase vengono esaminate le interazioni tra le principali classi, come quelle responsabili della gestione segnalazioni, autenticazione, notizie. L'integrazione seguirà un approccio *bottom-up*: si partirà dalle classi DAO per poi collegarle progressivamente ai servizi che le utilizzano.

3. **Test di sistema**, che ha lo scopo di valutare il comportamento complessivo di FakeNews Checker. Qui vengono riprodotti scenari realistici e modalità operative tipiche degli utenti, includendo funzionalità come registrazione, login, segnalazioni, verifica e gestione amministrativa. Le prove vengono eseguite in un ambiente che riproduce quello di produzione, così da far emergere eventuali criticità prima della distribuzione finale.

6. SUSPENSION AND RESUMPTION

La fase di testing del sistema FakeNews Checker verrà interrotta ogni volta che dovessero emergere condizioni tali da rendere impossibile il normale svolgimento delle attività previste.

Le principali cause che possono portare alla sospensione sono:

- **Individuazione di bug critici:** Qualsiasi errore che comprometta le funzionalità essenziali del sistema come crash improvvisi o problemi nel database, che renda impossibile proseguire con i test programmati.
- **Presenza di modifiche non ancora integrate:** Aggiornamenti al codice o nuove funzionalità che non siano state adeguatamente verificate o documentate e che potrebbero alterare il comportamento del sistema.

Per poter riprendere le attività di testing sarà necessario:

- **Risoluzione dei bug rilevati:** Una volta corrette le anomalie, verranno eseguiti test di regressione specifici per assicurarsi che gli interventi effettuati non abbiano introdotto ulteriori problemi.

7. TESTING MATERIALS

Per l'attività di testing del sistema FakeNews Checker verranno impiegati strumenti software dedicati, ciascuno con un ruolo specifico nel processo di verifica.

❖ Selenium

Selenium verrà adottato per automatizzare i test di sistema, permettendo di riprodurre le azioni tipiche degli utenti finali. L'esecuzione dei test automatizzati consentirà di controllare la correttezza delle interazioni tra utente e interfaccia, oltre a facilitare l'individuazione di eventuali anomalie o comportamenti anomali nell'interfaccia grafica.

❖ JUnit

JUnit rappresenterà il framework principale per il testing unitario. Sarà utilizzato per verificare il corretto funzionamento dei metodi appartenenti alle classi DAO, alle servlet e alle componenti che implementano la logica di business. I test unitari garantiranno che ogni singolo modulo rispetti i requisiti previsti e operi come atteso. Inoltre, JUnit fornirà report dettagliati utili per identificare rapidamente eventuali errori.

❖ Mockito

Mockito sarà impiegato sia nel testing unitario sia nel testing di integrazione. Sarà possibile simulare le interazioni tra le diverse componenti del sistema, come ad esempio tra DAO e database, oppure tra controller e servizi applicativi.

8. TEST CASES

I test case specifici associati a questa sezione sono riportati all'interno del documento **TCS_FakeNewsChcker**.

9. TESTING SCHEDULE

Il piano di testing previsto per il sistema FakeNews Chcker è stato organizzato con l'obiettivo di assicurare un processo di verifica accurato, coerente e perfettamente sincronizzato con le fasi di sviluppo. L'intero ciclo di test è articolato in tre momenti distinti, ciascuno dedicato a un diverso livello del sistema:

- **Testing Unitario:** In questa fase vengono controllate singolarmente le classi e i metodi implementati, così da garantire che ogni unità rispetti i requisiti funzionali. Le attività saranno svolte parallelamente allo sviluppo, in modo da ridurre i tempi complessivi e individuare tempestivamente eventuali anomalie.
- **Testing di Integrazione:** Questa fase si concentra sulla verifica delle interazioni tra i vari moduli, includendo lo scambio di dati tra servlet e servizi e il controllo delle dipendenze tra componenti. L'obiettivo è assicurarsi che i moduli collaborino correttamente una volta combinati.
- **Testing di Sistema:** Qui viene valutato il comportamento dell'intera applicazione, analizzando sia le funzionalità sia le prestazioni. Verranno riprodotti scenari d'uso realistici.

Al termine dello sviluppo, l'intero set di test verrà eseguito nuovamente come verifica conclusiva, così da confermare la stabilità del sistema e produrre la documentazione finale. Ogni fase sarà seguita da una revisione dettagliata, utile a correggere eventuali problemi prima di procedere allo step successivo, garantendo un rilascio finale privo di bug critici.

10. DEFINITIONS, ACRONYMS AND ABBREVIATIONS

DAO	Data access object, sono gli oggetti utilizzati per accedere ai dati persistenti
RAD	Documento di analisi dei requisiti
ODD	Documento di Object Design